Lecture 9

Parallel Computing

- Instruction-Level Parallelism (ILP)

- Thread-Level Parallelism (TLP)

Modifications to the von Neumann model

- Since the first electronic digital computers were developed back in the 1940s, computer scientists and computer engineers have made many improvements to the basic von Neumann architecture.
- Many are targeted at reducing the problem of the von Neumann bottleneck, but many are also targeted at simply making CPUs faster.
- In the last lecture, we talked about caching, and now we will talk about another improvements methods:

  - Instruction-Level Parallelism (ILP)

  - Thread-Level Parallelism (TLP)

A. Instruction-level parallelism (ILP)

- Instruction level parallelism, or ILP, attempts to improve processor performance by having multiple processor components or functional units simultaneously executing instructions.
- There are two main approaches to ILP: pipelining, in which functional units are arranged in stages; and multiple issue (Superscalar) processor, in which multiple instructions can be simultaneously initiated.
- Both approaches are used in virtually all modern CPUs.

1. Pipelining

- *Pipelining* is an implementation technique whereby multiple instructions are overlapped in execution.
- If the system repeatedly executes a basic function, it divides the execution of an operation into a sequential independent stages.
- Pipelines improve performance by taking individual pieces of hardware or functional units and connecting them in sequence.
- In a computer pipeline, each stage in the pipeline completes a part of an instruction.
- The stages are connected one to the next to form a pipe through a register for synchronization.

Speedup

- The throughput of an instruction pipeline is determined by how often an instruction exits the pipeline.
- Let a pipeline of depth k, can process n items in ‘n+ k’ steps (throughput), where, without pipeline it takes ‘n x k’, therefore, Speedup = (n x k) / (n + k)

Example:

Add 100 pairs of numbers, one Add takes 5 cycles, i.e., k = 5, n = 100.

Speedup = 5

Example

- Suppose we want to add the floating point numbers:

  9.87×10^4 and 6.54×10^3

- Then we can use the following steps:
- In the example, normalizing shifts the decimal point one unit to the left, and rounding rounds to three digits.

Time	Operation	Operand 1	Operand 2	Result
1	Fetch operands	9.87×10^4	6.54×10^3	
2	Compare exponents	9.87×10^4	6.54×10^3	
3	Shift one operand	9.87×10^4	0.654×10^4	
4	Add	9.87×10^4	0.654×10^4	10.524×10^4
5	Normalize result	9.87×10^4	0.654×10^4	1.0524×10^5
6	Round result	9.87×10^4	0.654×10^4	1.05×10^5
7	Store result	9.87×10^4	0.654×10^4	1.05×10^5

Table 2.3 Pipelined addition. Numbers in the table are subscripts of operands/results.

Time	Fetch	Compare	Shift	Add	Normalize	Round	Store
0	0						
1	1	0					
2	2	1	0				
3	3	2	1	0			
4	4	3	2	1	0		
5	5	4	3	2	1	0	
6	6	5	4	3	2	1	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
999	999	998	997	996	995	994	993
1000		999	998	997	996	995	994
1001			999	998	997	996	995
1003					999	998	997
1004						999	998
1005							999

Cont.,

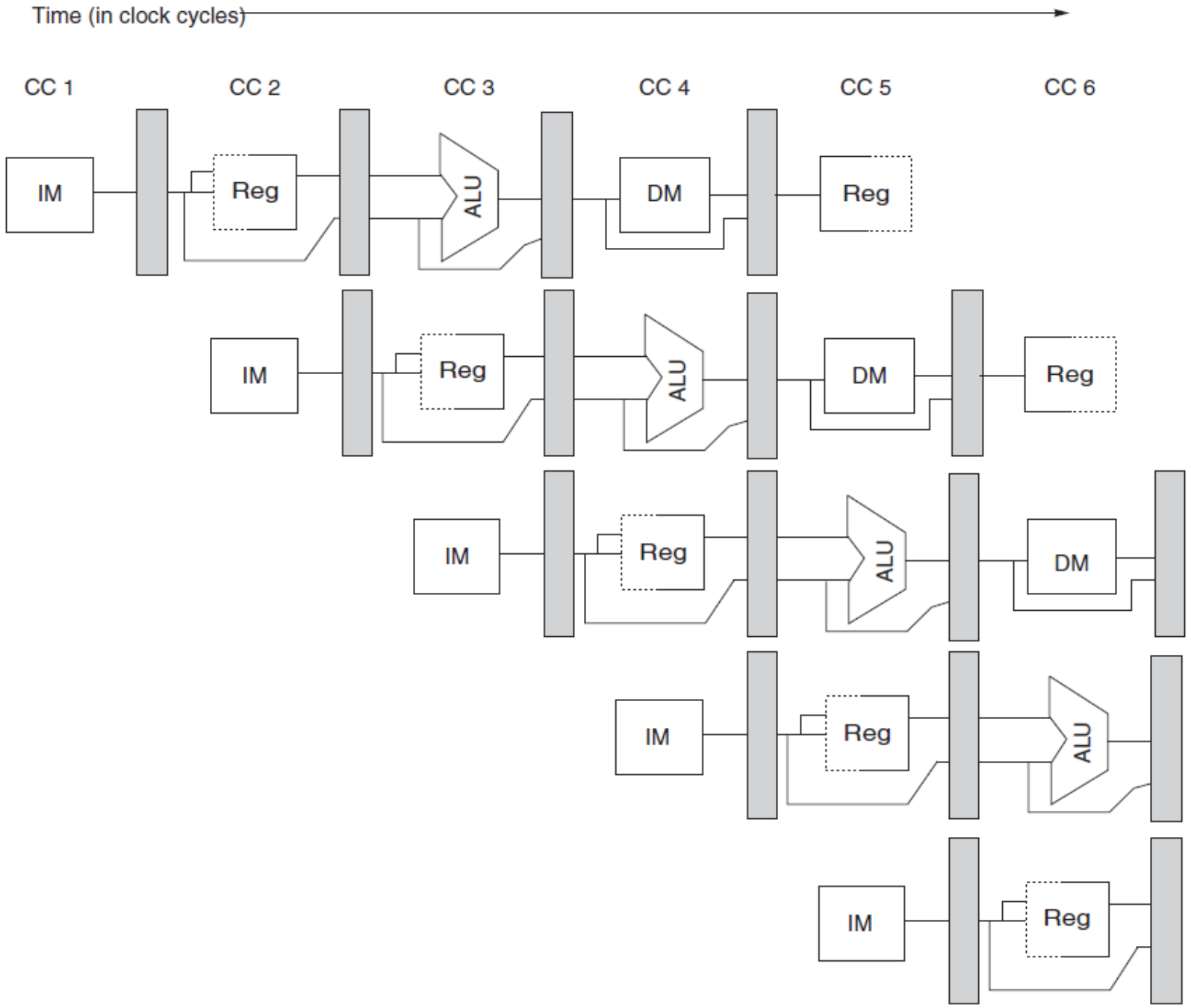
- So if we execute the code:

```
float x [1000] , y [1000] , z [1000];
...
for ( i = 0; i < 1000; i ++)
    z [ i ] = x [ i ] + y [ i ] ;
```

- If each of the operations takes one nanosecond, then the for loop will take something like 7000 nanoseconds.
- As an alternative, suppose we divide our floating point adder into seven separate pieces of hardware or functional units.
- The total time to execute the for loop has been reduced from 7000 nanoseconds to 1006 nanoseconds—an improvement of almost a factor of seven

Pipeline Registers

- It is critical to ensure that instructions in the pipeline do not attempt to use the hardware resources at the same time, and also ensure that instructions in different stages of the pipeline do not interfere with one another.
- This separation is done by introducing *pipeline registers* between successive stages of the pipeline.
- So that at the end of a clock cycle all the results from a given stage are stored into a register that is used as the input to the next stage on the next clock cycle.


Time (in clock cycles)
CC 1
CC 2
CC 3
CC 4
CC 5
CC 6
IM
Reg
ALU
DM
Reg
IM
Reg
ALU
DM
Reg
IM
Reg
ALU
DM
IM
Reg
ALU
IM
Reg

Pipeline Hazards

- There are situations, called *hazards*, that prevent the next instruction in the instruction stream from executing during its designated clock cycle.
- Hazards reduce the performance from the ideal speedup gained by pipelining.
- There are three classes of hazards:

  1. *Structural hazards* arise from resource conflicts when the hardware cannot support all possible combinations of instructions simultaneously in overlapped execution.

  2. *Data hazards* arise when an instruction depends on the results of a previous instruction .

  3. *Control hazards* arise from the pipelining of branches and other instructions that change the PC.

Cont.,

- Hazards in pipelines can make it necessary to *stall* the pipeline.
- Instructions issued *earlier* than the stalled instruction—and hence farther along in the pipeline—must continue, since otherwise the hazard will never clear.
- As a result, no new instructions are fetched during the stall.

1. Structural Hazards

- When a processor is pipelined, the overlapped execution of instructions requires pipelining of functional units and duplication of resources to allow all possible combinations of instructions in the pipeline.
- Structural hazards arise when some functional unit is not fully pipelined.
- Another common way that structural hazards appear is when some resource has not been duplicated enough to allow all combinations of instructions in the pipeline to execute.

Example: a load with one memory port

- Some pipelined processors have shared a single-memory pipeline for data and instructions.
- As a result, when an instruction contains a data memory reference, it will conflict with the instruction reference for a later instruction, as shown in Figure C.4.
- To resolve this hazard, we stall the pipeline for 1 clock cycle when the data memory access occurs.
- *For the following figure, the load instruction uses the memory for a data access, and at the same time instruction 3 wants to fetch an instruction from memory.*

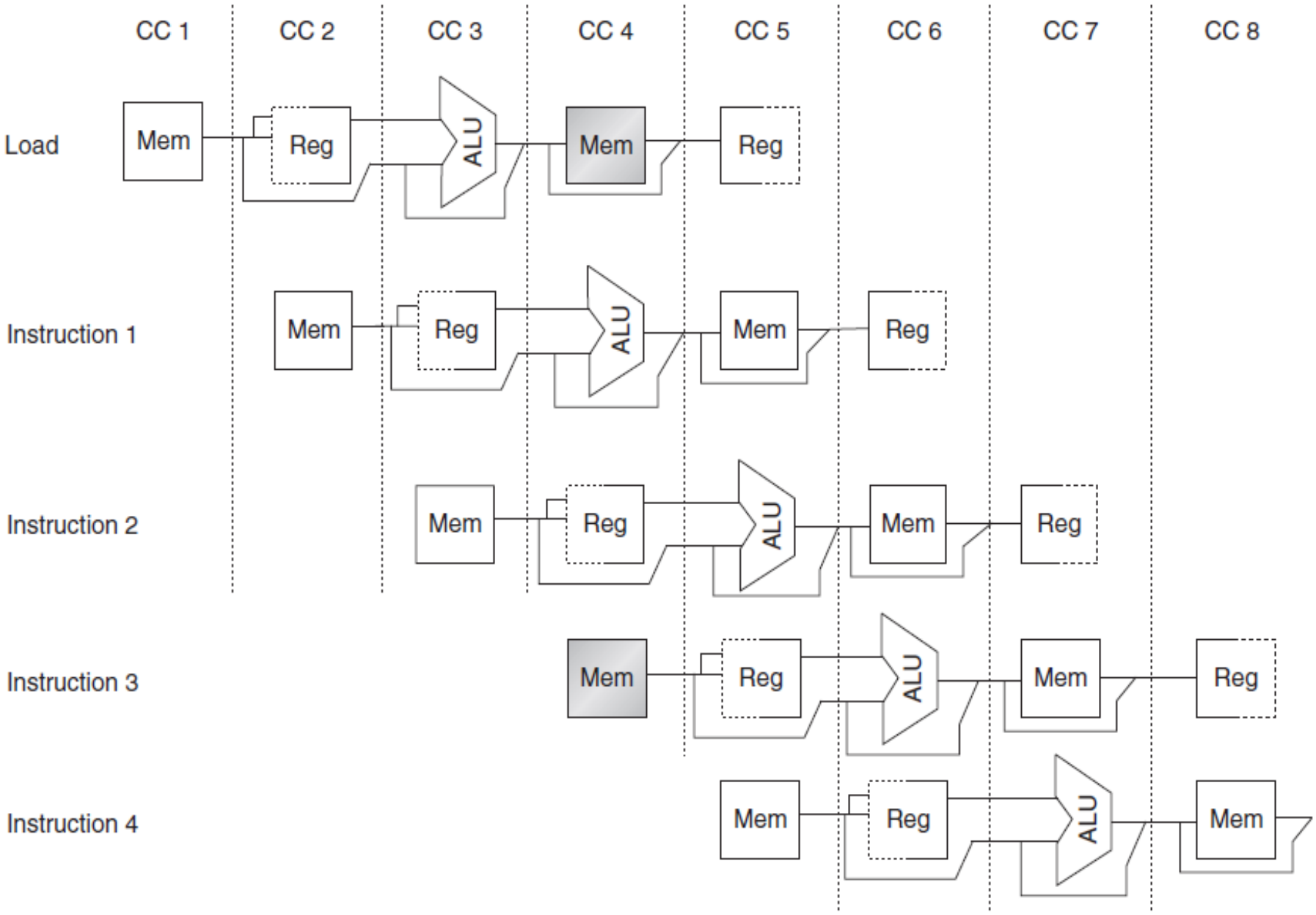


Figure C.4 A processor with only one memory port will generate a conflict whenever a memory reference occurs. In this example the load instruction uses the memory for a data access at the same time instruction 3 wants to fetch an instruction from memory.

	Clock cycle number									
Instruction	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i+1$		IF	ID	EX	MEM	WB				
Instruction $i+2$			IF	ID	EX	MEM	WB			
Instruction $i+3$				Stall	IF	ID	EX	MEM	WB	
Instruction $i+4$						IF	ID	EX	MEM	WB
Instruction $i+5$							IF	ID	EX	MEM
Instruction $i+6$								IF	ID	EX

2- Data Hazards

Consider the pipelined execution of these instructions:

I1: ADD R1,R2,R3

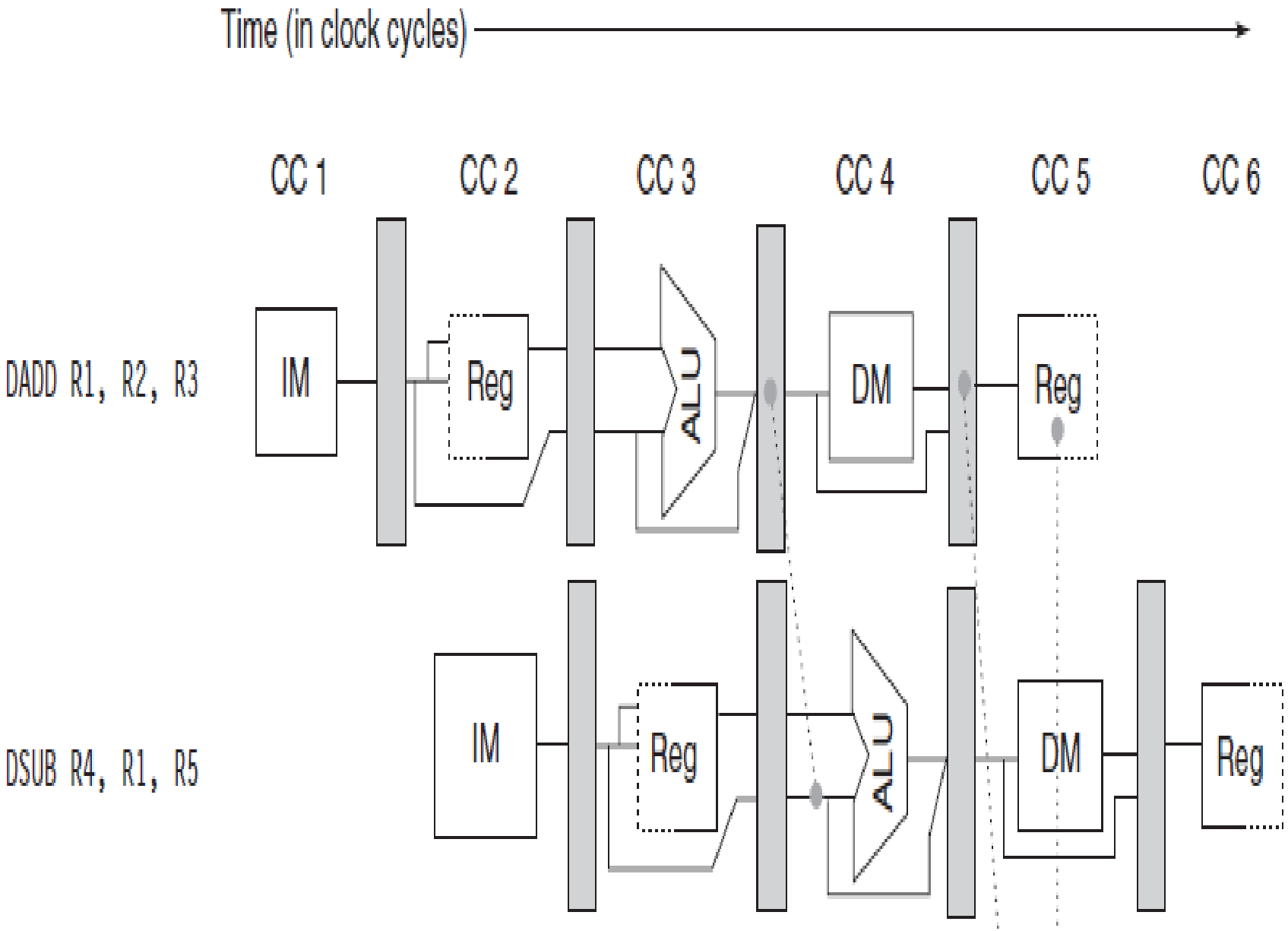
I2: SUB R4,R1,R5

Where, R1 is the output of I1, and input in I2.

Solution:

By using Forwarding (Bypassing) hardware.

When it detects that the previous ALU operation has the source for the current ALU operation, control logic (MUX) selects the forwarded result as an ALU input rather than the value read from the register file.


Time (in clock cycles)
CC 1
CC 2
CC 3
CC 4
CC 5
CC 6
DADD R1, R2, R3
IM
Reg
ALU
DM
Reg
DSUB R4, R1, R5
IM
Reg
ALU
DM
Reg

3- Control (Branch) Hazards (sol: Branch Prediction)

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

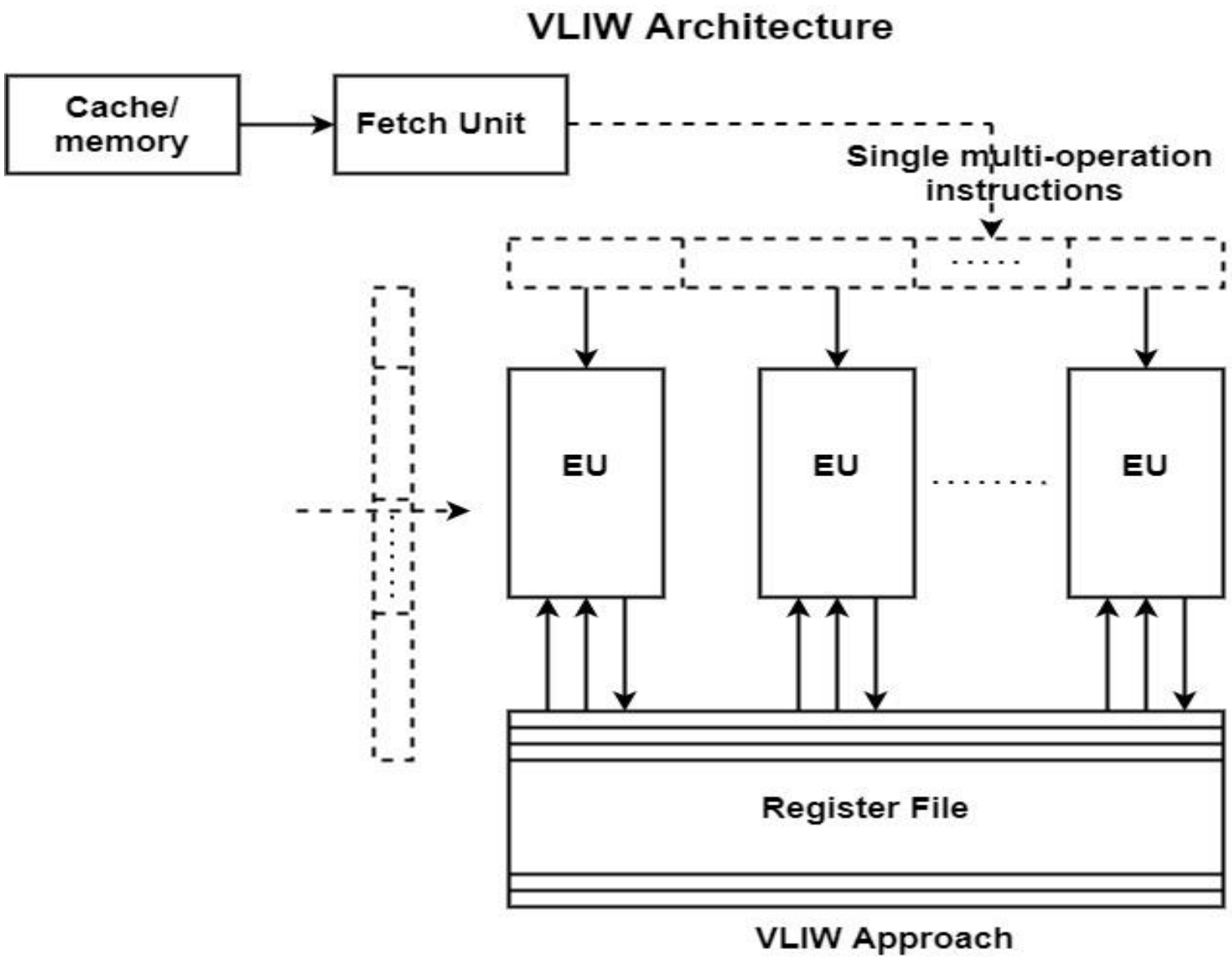
Figure C.11 A branch causes a one-cycle stall in the five-stage pipeline. The instruction after the branch is fetched, but the instruction is ignored, and the fetch is restarted once the branch target is known. It is probably obvious that if the branch is not taken, the second IF for branch successor is redundant. This will be addressed shortly.

2. Multiple issue (Superscalar) Processor

- Pipelines improve performance by taking individual pieces of hardware or functional units and connecting them in sequence.
- Multiple issue processors replicate functional units and try to simultaneously execute different instructions in a program.
- For example, if we have two complete floating point adders, we can approximately halve the time it takes to execute the loop

Very Long Instruction Word (VLIW) Architecture

- It is an instruction set architecture constructed to take complete benefit of instruction-level parallelism (ILP) for upgraded performance.
- VLIW architectures are closely associated with superscalar processors.
- Both objectives at speeding up computation with the aid of exploiting instruction-level parallelism.
- Both have almost a similar execution basis, including various execution units (EUs) controlling in parallel.


VLIW Architecture
Cache/ memory
Fetch Unit
Single multi-operation instructions
EU
EU
EU
Register File
VLIW Approach

Instruction Issue Methods

- The processor's policy toward Fetching, Decoding, and execution instructions have a significant effect on its ability to discover parallelism.
- Instruction Issue is the process of initiating instruction execution in functional units.

    1- In-Order issue and In-order completion,

    2- In-Order issue and Out-of order completion.

    3- Out-of-Order issue and Out-of -order completion

Static & Dynamic Multiple Issue

- If the functional units are *scheduled* at compile time, the multiple issue system is said to use static multiple issue.
- The system is said to use dynamic multiple issue if they're *scheduled* at run-time, A processor that supports dynamic multiple issue is sometimes said to be superscalar.
- Of course, to make use of multiple issue, the system must find instructions that can be executed simultaneously.
- One of the most important techniques is speculation.
- In speculation, the compiler or the processor makes a guess about an instruction, and then executes the instruction on the basis of the guess.

Speculation

- If the compiler does the speculation, it will usually insert code that tests whether the speculation was correct, and, if not, takes corrective action.
- If the hardware does the speculation, the processor usually stores the result(s) of the speculative execution in a buffer.
- When it's known that the speculation was correct, the contents of the buffer are transferred to registers or memory.
- If the speculation was incorrect, the contents of the buffer are discarded, and the instruction is re-executed.

Example

- In the following code, the system might predict that the outcome of z = x + y will give z a positive value, and, as a consequence, it will assign w = x:

```
z = x + y;
if (z > 0)
    w = x;
else
    w = y;
```

- We will need to go back and execute the assignment w = y if the assignment z = x + y results in a value that's not positive.

Dependences

- Determining how one instruction depends on another is critical to determining how much parallelism exists in a program and how that parallelism can be exploited.
- In particular, to exploit instruction-level parallelism we must determine which instructions can be executed in parallel.
- If two instructions are parallel, they can execute simultaneously in a ILP without causing any stalls, assuming the system has sufficient resources.
- If two instructions are dependent, they are not parallel and must be executed in order.

Cont.,

- There are three different types of dependences:

  1- Data dependences,

  2- Resource dependences,

  3- Control dependences.

1- Data Dependences

- Flow dependence → Statement S2 is flow-dependent on S1, if at least one output of S1 is an input to S2.

- Antidependence →

  If S2 follow S1 in a program order, and the output of S2 overlaps the input to S1.

- An output dependence → occurs when instruction i and instruction j write the same register or memory location.

Example

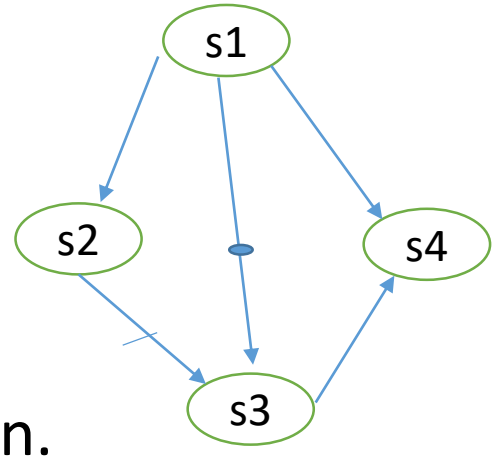
- Consider the following code, and detect dependences:

S1: LD R1, A

S2: ADD R2, R1

S3: MOV R1, R3

S4: ST B, R1



Where:

- The dependence graph as shown.
- S4 needs R1 which can be from S1 or S3.
- In case of out-of-order-completion, S4 can be completed before S3, and it will be not correct.

Cont.,

- 2. Resource Dependence →Is concerned with the conflicts in using shared resources such as: Registers, Memory area, Function units (ALU). The symbol ⇢ refers to shared resources.
- 3. Control Dependences → It determines the ordering of an instruction, i, with respect to a branch instruction so that instruction i is executed in correct program order and only when it should be. For example, in the code segment:

```
if p1 {
        S1;
};
if p2 {
        S2;
}
```

S1 is control dependent on p1, and S2 is control dependent on p2 but not on p1.

IA-64 computer Machine

- The architecture is based on explicit instruction-level parallelism, in which the compiler decides which instructions to execute in parallel.
- This contrasts with superscalar architectures, which depend on the processor to manage instruction dependencies at runtime.
- Cores execute up to six instructions per cycle.
- The processor has thirty functional execution units in eleven groups.

B. Thread-level parallelism (TLP)

- ILP can be very difficult to exploit: a program with a long sequence of dependent statements offers few opportunities.
- For example, in a direct calculation of the Fibonacci numbers:

```
f[0] = f[1] = 1;
for (i = 2; i <= n; i++)
    f[i] = f[i-1] + f[i-2];
```

- There's essentially no opportunity for simultaneous execution of instructions. (Why?)
- Thread-level parallelism, or TLP, attempts to provide parallelism through the simultaneous execution of different threads

Multitasking

- Multitasking is the process of scheduling and switching the CPU between several tasks.
- OR, a single CPU switches its attention between several sequential tasks.
- It maximizes the utilization of the CPU and also provides modular construction of application.
- A Real time operating system is a multitasking operating system;
- **Types of RTOS**:
- **Non Preemptive RTOS**: in this type of RTOS, new higher priority task will gain control of the CPU only when the current task gives up the CPU.
- **Preemptive RTOS** In this type of RTOS, if any higher priority task or ISR is ready to run, the current task is preempted (suspended) and higher task is immediately given the control of CPU.

Threading

- Threading provides a mechanism for programmers to divide their programs into more or less independent tasks, with the property that when one thread is blocked, another thread can be run.
- Furthermore, in most systems it’s possible to switch between threads much faster than it’s possible to switch between processes.
- This is because threads are “lighter weight” than processes.
- In fact, two threads belonging to one process can share most of the process’s resources.

Cont.,

- If a process is the “master” thread of execution and threads are started and stopped by the process, then we can envision the process and its subsidiary threads as lines.
- When a thread is started, it forks off the process; when a thread terminates, it joins the process.

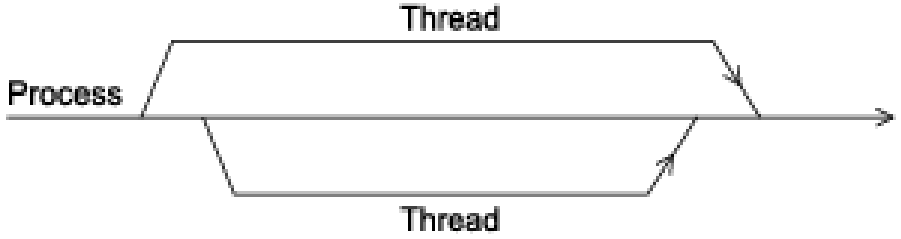


FIGURE 2.2

A process with three threads.

Hardware multithreading

- It provides a means for systems to continue doing useful work when the task being currently executed has stalled—for example, if the current task has to wait for data to be loaded from memory.
- Instead of looking for parallelism in the currently executing thread, it may make sense to simply run another thread.
- Of course, for this to be useful, the system must support very rapid switching between threads.

Fine Grain Multithreading

- Fine grain refers to a high level of granularity, where tasks or resources are divided into very small units.
- **Characteristics**:
  - **Tasks**: Smaller, more frequent tasks that can be executed independently.
  - **Control**: Allows for more detailed control over resources, enabling dynamic allocation and scheduling.
  - **Example**: In parallel computing, fine-grained parallelism involves splitting a task into many small threads that run simultaneously, like in multi-threaded applications.
  - **Overhead**: Typically incurs higher overhead due to frequent context switching and communication between threads.

Coarse Grain Multithreading

- Coarse grain refers to a lower level of granularity, where tasks or resources are divided into larger units.
- **Characteristics**:
  - **Tasks**: Larger, less frequent tasks that may be less independent.
  - **Control**: Offers less detailed control but reduces overhead and can simplify design.
  - **Example**: Coarse-grained parallelism might involve dividing a workload into a few large threads or processes that run concurrently.
  - **Efficiency**: Generally more efficient in terms of resource usage, as it reduces the overhead associated with managing many small tasks.

Fine-Grain Vs., Coarse Grain

- **Parallel Computing**: Fine grain is often used for applications requiring high parallelism, while coarse grain is preferred for tasks where communication overhead between tasks is significant.
- **Resource Management**: In operating systems, fine-grained resource management allows for better optimization but can lead to complexity, whereas coarse-grained management is simpler but might not utilize resources as efficiently.
- In fine-grained multithreading, the processor switches between threads after each instruction, skipping threads that are stalled.

Cont.,

- However, it has the drawback that a thread that's ready to execute a long sequence of instructions may have to wait to execute every instruction.
- Coarse-grained multithreading attempts to avoid this problem by only switching threads that are stalled waiting for a time-consuming operation to complete (e.g., a load from main memory).
- In summary, the choice between fine grain and coarse grain approaches depends on the specific requirements of the application, including performance needs, complexity, and resource management.

Simultaneous Multithreading (SMT)

- SMT, is a variation on fine-grained multithreading.
- It attempts to exploit superscalar processors by allowing multiple threads to make use of the multiple functional units.
- If we designate “preferred” threads— threads that have many instructions ready to execute—we can somewhat reduce the problem of thread slowdown.

Threads suffer from: 1- Race-conditions

- Two concurrent pieces of code race to access the same resource.
- Example: Assume that two threads each increment a global variable by one. If the two threads run simultaneously without locking or without synchronization, the outcome of the operation could be wrong.

Thread 1	Thread 2		Integer value
			0
read value		←	0
	read value	←	0
increase value			0
	increase value		0
write back		→	1
	write back	→	1

Solution: Mutual Exclusion (Mutex) Lock

- A phenomenon for solving the shared data problem (Race condition) is known as semaphore.
- Mutex is a semaphore that gives at an instance two tasks mutually exclusive access to resources.
- A mutual exclusion lock prevents any two threads from simultaneously accessing or modifying a shared resource.
- The code between the lock and unlock is a critical section.
- At any one time, only one thread can be executing code in such a critical section.
- A programmer may need to ensure that all accesses to a shared resource are similarly protected by locks.

2- Deadlock

- A deadlock occurs when some threads become permanently blocked trying to acquire locks.
- Deadlock is the name given to the condition where two or more processes are blocked waiting for each other to unlock critical section of codes.
- This can occur, for example, if thread A holds lock1 and then blocks trying to acquire lock2, which is held by thread B, and then thread B blocks trying to acquire lock1.
- Such deadly embraces have no clean escape. The program needs to be aborted.

3- Memory consistency

- Which defines how variables that are read and written by different threads appear to those threads.
- Intuitively, reading a variable should yield the last value written to the variable, but what does “last” mean?
- Strict consistency: A read operation shall return the value written by the most recent operation.
- Sequential consistency: means that the result of any execution is the same as if the operations of all threads are executed in some sequential order, and the operations of each individual thread appear in this sequence in the order specified by the thread.

Assignment

- When we were discussing floating point addition, we made the simplifying assumption that each of the functional units took the same amount of time. Suppose that fetch and store each take 2 nanoseconds and the remaining operations each take 1 nanosecond.

  a. How long does a floating point addition take with these assumptions?

  b. How long will an unpipelined addition of 1000 pairs of floats take with these assumptions?

  c. How long will a pipelined addition of 1000 pairs of floats take with these assumptions?